

Sistema de Aprovação de Notas Fiscais — Manual de Implantação

Pronep Life Care

Versão 1.0 — Maio/2026

Sumário

Parte I — Visão Geral

Apresentação do sistema

Arquitetura técnica

Pré-requisitos

Parte II — Implantação passo a passo

Microsoft Entra ID (App Registration + Grupos + Permissões)

SharePoint (Site + 5 Listas + Pastas)

Azure Static Web App (Provisionamento + Configuração)

GitHub (Repositório + CI/CD)

Teams App (Manifest + Instalação)

Notificações Push (VAPID + opcional)

Parte III — Referência técnica

Estrutura do projeto

Catálogo de Azure Functions

Módulos compartilhados

Frontend (SPA)

Variáveis de ambiente (referência completa)

Parte IV — Operação

Fluxos do sistema

Perfis de usuário

Notificações

Backups e auditoria

Parte V — Anexos

Checklist de implantação

Troubleshooting

Mapa de permissões Graph

Glossário

Parte I — Visão Geral

1. Apresentação do sistema

O Sistema de Aprovação de Notas Fiscais (Sistema NF) é uma aplicação web corporativa desenvolvida pela Pronep Life Care para padronizar e digitalizar o processo de lançamento, aprovação, rejeição e arquivamento de notas fiscais de serviço.

A aplicação substitui o fluxo manual baseado em Power Automate e e-mails, oferecendo:

- **Tela única e moderna** para lançamento de NF com upload de PDF
- **Roteamento automático** de aprovadores conforme a Unidade e a Diretoria do fornecedor
- **Aprovação com marca d'água** automática no PDF (rastreadabilidade visual)
- **Notificações em 3 canais** simultâneos: e-mail, Microsoft Teams e push nativo (mobile/desktop)
- **Multi-nível de aprovação** por valor (ex.: NFs acima de R\$ 30 mil exigem aprovação de gestor master)
- **Deteção de duplicidade** por hash SHA-256 do arquivo + chave CNPJ/Número/Série
- **Cadastro centralizado de fornecedores** com importação em massa via planilha
- **Dashboard analítico** com KPIs, gráficos e filtros por unidade
- **PWA (Progressive Web App)** instalável em Windows, Android e iOS
- **RBAC (controle de acesso por perfil)** integrado ao Microsoft Entra ID

Stack tecnológica

Camada	Tecnologia
Hospedagem	Azure Static Web Apps (Standard SKU)
Backend	Azure Functions (Node.js v22)
Autenticação	Microsoft Entra ID (Easy Auth) + Teams SSO
Persistência	Microsoft SharePoint (Listas + Document Library)
Notificações	Microsoft Graph API (Mail.Send + sendActivityNotification) + Web Push (VAPID)
Frontend	HTML/CSS/JavaScript vanilla (SPA)
Bibliotecas frontend	Chart.js, Microsoft Teams JS SDK, SheetJS
Bibliotecas backend	@azure/identity, @microsoft/microsoft-graph-client, pdf-lib, jsonwebtoken, web-push
CI/CD	GitHub Actions
Embarque em Teams	Personal Tab (Teams App manifest 1.16)

Custos

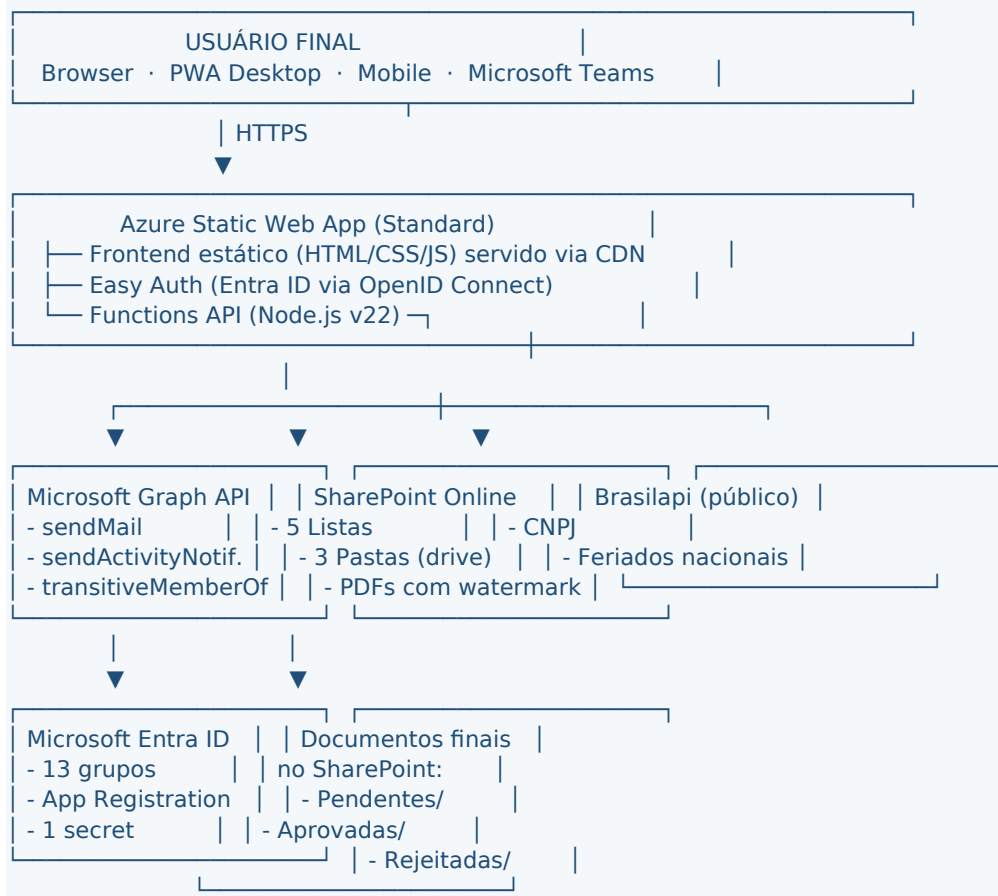
Toda a stack roda sobre licenças que a Pronep já possui:

Recurso	Plano	Custo adicional
Azure Static Web Apps	Standard	~US\$ 9/mês (necessário para Teams SSO; Free não suporta)
Microsoft 365 (Entra ID, SharePoint, Teams)	já licenciado	—
GitHub	já licenciado	—
Brasilapi (CNPJ + feriados)	público	—

O Standard SKU do SWA é obrigatório porque o Teams SSO exige Custom Authentication via App Registration própria, recurso não disponível no Free.

2. Arquitetura técnica

Diagrama de alto nível



Componentes principais

Azure Static Web App — entrypoint único. Hospeda o frontend (HTML/CSS/JS vanilla) e os 22 endpoints da API (Azure Functions). Tem Easy Auth nativo integrado com Entra ID via OpenID Connect.

Microsoft Entra ID — autenticação e autorização. O sistema utiliza:

- 1 App Registration (com Client ID + Secret) para chamadas Application-mode ao Graph
- 13 Grupos de segurança que mapeiam perfis (admin, financeiro, gestores por diretoria, solicitantes)
- Federated identity opcional para Teams SSO

SharePoint — persistência. 5 Listas atuam como banco de dados, 3 Pastas no Document Library armazenam os PDFs.

Microsoft Graph API — porta única para todas as integrações com o ecossistema Microsoft (envio de e-mail, notificações Teams, leitura de membros de grupo).

GitHub — repositório de código e CI/CD via Actions. Cada push na branch main aciona deploy automático.

Fluxo de aprovação de NF (visão simplificada)

Solicitante abre o app → menu **Lançar NF** → seleciona fornecedor (auto-resolve unidade e diretoria) → preenche número, série, valor, vencimento → faz upload do PDF.

Backend (PostNota) calcula hash SHA-256 → consulta lista de NFs por duplicidade → consulta matriz Diretorias para resolver o e-mail do aprovador → grava PDF em Notas Fiscais/Pendentes/{Unidade}/Diretoria {Diretoria}/ → cria item na lista PRONEP-NF-NotasFiscais com Status=Lancada.

Notificações são disparadas em paralelo: e-mail HTML para o aprovador (com botões Aprovar/Rejeitar assinados via JWT), notificação 1:1 no Microsoft Teams via sendActivityNotification e push notification para dispositivos cadastrados.

Aprovador clica no botão do e-mail (sem precisar logar), ou abre o app → **Fila de Aprovação** → escolhe Aprovar ou Rejeitar.

Backend (AprovarNota) consulta a config global para verificar se valor exige 2º nível. Se sim, encaminha para o Gestor Master. Senão, baixa o PDF, aplica marca d'água "APROVADO" + data + e-mail do aprovador via pdf-lib, move o PDF para Notas Aprovadas/{Unidade}/{AAAA-MM-DD}/, atualiza o item da lista para Status=Aprovada e envia notificações de fechamento.

Setor financeiro marca o checkbox "Processado" na tela de Notas Aprovadas quando integra a NF ao sistema fiscal.

3. Pré-requisitos

Acessos necessários

Antes de iniciar a implantação, garanta os seguintes acessos no tenant onde o sistema vai rodar:

Recurso	Permissão mínima	Para que serve
Microsoft Entra ID	Application Administrator + Groups Administrator	Criar App Registration, atribuir permissões Graph com consentimento de admin, criar grupos de segurança
Azure Subscription	Contributor (na Subscription ou no Resource Group destino)	Criar o Static Web App
SharePoint Online	Site Collection Administrator (no site usado)	Criar listas, configurar permissões, gerenciar Document Library
Microsoft Teams Admin Center	Teams Administrator	Fazer upload do pacote ZIP do Teams App e liberar política de instalação
GitHub	Admin do repositório	Configurar secrets do Actions e fazer push da branch main

Conta de serviço para envio de e-mails

O sistema usa Graph API com permissão **Mail.Send** (Application). Toda mensagem é enviada como se fosse de uma caixa específica. Recomendação:

- Criar (ou reaproveitar) uma conta dedicada, **sem MFA interativo bloqueante**, ex.: datanalytics@pronep.com.br ou sistema-nf@pronep.com.br
- A conta precisa ter licença que inclua Exchange Online
- Configurar o endereço dessa conta no App Setting EMAIL_FROM_ADDRESS

Ferramentas locais (na máquina do administrador que vai implantar)

Ferramenta	Versão mínima	Onde baixar
Git	qualquer recente	git-scm.com
Node.js	18.x ou superior	nodejs.org
Python	3.10+ (apenas para gerar VAPID keys uma única vez)	python.org
Editor de texto/IDE	VS Code recomendado	code.visualstudio.com

Navegador moderno	Edge ou Chrome	—

Parte II — Implantação passo a passo

4. Microsoft Entra ID

Esta etapa cria a identidade da aplicação no Entra ID, define quem terá acesso e libera as permissões necessárias para o sistema interagir com Graph, SharePoint e Teams.

4.1 Criar a App Registration

Acessar https://portal.azure.com → buscar **Microsoft Entra ID** → menu lateral **App registrations** → **+ New registration**.

Preencher:

- **Name:** Pronep Aprovacao NF SWA (ou nome similar; será o display name)
- **Supported account types:** *Accounts in this organizational directory only (Single tenant)*
- **Redirect URI:** deixar em branco neste momento (será preenchido após criar o Static Web App)

Clicar em **Register**.

Anotar dois valores da tela **Overview**:

- **Application (client) ID** — será usado como AAD_CLIENT_ID
- **Directory (tenant) ID** — será usado como AAD_TENANT_ID

4.2 Criar um client secret

Na App Registration recém-criada, menu lateral **Certificates & secrets** → aba **Client secrets** → **+ New client secret**.

Preencher:

- **Description:** SWA Functions secret
- **Expires:** 24 months (recomendado; agendar renovação)

Clicar **Add**.

Copiar imediatamente o valor (campo Value) — este valor só aparece uma vez. Ele será usado como AAD_CLIENT_SECRET. Guardar em um cofre de senhas corporativo (1Password, Bitwarden, KeePass, etc.).

Atenção: o Client Secret é equivalente a uma senha de serviço. Nunca commitar em código nem colar em documentos versionados. Apenas em App Settings do Azure ou cofre de senhas.

4.3 Habilitar Implicit grant e Token configuration

Menu lateral **Authentication** → seção **Implicit grant and hybrid flows** → marcar **ID tokens (used for implicit and hybrid flows)**. *Não marcar* "Access tokens" (mais seguro).

Salvar.

Menu lateral **Token configuration** → **+ Add groups claim**:

- Selecionar **Security groups**
- Em ID, Access e SAML: manter **Group ID** marcado
- Salvar

Esse passo garante que os tokens de ID emitidos vão conter os GUIDs dos grupos aos quais o usuário pertence — sem isso, o sistema não consegue mapear perfis.

4.4 Adicionar permissões da Microsoft Graph

Menu lateral **API permissions** → **+ Add a permission** → **Microsoft Graph** → **Application permissions** (não Delegated).

Adicionar uma por uma as permissões abaixo:

Permissão	Por que o sistema precisa
GroupMember.Read.All	Resolver perfil do usuário a partir dos grupos do Entra ID (MeusGrupos)
User.Read.All	Resolver email → objectId do Entra ID para enviar notificação Teams
Sites.ReadWrite.All	Ler, criar e atualizar itens nas Listas e Document Library do SharePoint
Mail.Send	Enviar e-mails transacionais a partir da caixa configurada em EMAIL_FROM_ADDRESS
TeamsActivity.Send	Enviar notificações 1:1 no Teams via sendActivityNotification
TeamsAppInstallation.ReadWriteForUser.All	Instalar o Teams App automaticamente para o aprovador (pré-requisito do envio de Activity)
AppCatalog.Read.All	Descobrir o catalogAppId do Teams App publicado

Após adicionar todas, clicar em **Grant admin consent for [tenant]** no topo da tabela.

Confirmar que cada linha mostra status **Granted for [tenant]** com check verde.

Se aparecer "Not granted" mesmo após o clique, isso significa que o usuário atual não tem perfil de Application/Global Administrator. Solicitar ao admin do tenant.

4.5 Configurar Expose an API (para Teams SSO)

Esta etapa só é necessária se a empresa planeja usar o sistema embarcado dentro do Microsoft Teams como Personal Tab. Se não, pode ser pulada.

Menu lateral **Expose an API** → **Set** ao lado de "Application ID URI".

Substituir o valor sugerido por:

api://<HOST_DO_SWA>/<CLIENT_ID>

Exemplo: `api://aprovacao-nf-pronep.azurestaticapps.net/85f7fa68-a241-4caf-803f-9991bd1f0eee`

Salvar. Anotar este valor — será o APP_ID_URI.

Clicar **+ Add a scope**:

- **Scope name:** access_as_user
- **Who can consent:** Admins and users
- **Admin consent display name:** Acessar Sistema NF como o usuário
- **Admin consent description:** Permite que o Sistema de Aprovação de NF acesse dados do usuário via Teams SSO
- **User consent display name:** Acessar o Sistema de Aprovação de NF
- **User consent description:** Permite que o sistema acesse seus dados ao abrir a tab no Teams
- **State:** Enabled

Adicionar **Authorized client applications** (3 GUIDs fixos da Microsoft, que representam os clientes Teams):

- 1fec8e78-bce4-4aaf-ab1b-5451cc387264 (Microsoft Teams desktop e mobile)
- 5e3ce6c0-2b1f-4285-8d4b-75ee78787346 (Microsoft Teams web)
- 4765445b-32c6-49b0-83e6-1d93765276ca (Microsoft Teams Office app web)

Para cada um: **+ Add a client application** → cole o GUID → marque o scope access_as_user criado → **Add application**.

4.6 Criar os 13 grupos de segurança

O sistema usa grupos do Entra ID para definir perfis. Criar **todos** os 13 grupos abaixo (mesmo nomes literais):

Nome do grupo	Perfil mapeado	Tipo
PRONEP-NF-Submitter	Solicitante (lança NF)	Security
PRONEP-NF-Admin	Administrador do sistema	Security
PRONEP-Financeiro-Gestao	Financeiro (vê tudo, aprova como 2º nível)	Security
PRONEP-NF-Gestor-Suprimentos	Gestor da diretoria de Suprimentos	Security
PRONEP-NF-Gestor-Financeira	Gestor da diretoria Financeira	Security
PRONEP-NF-Gestor-Tecnologia	Gestor da diretoria de Tecnologia	Security
PRONEP-NF-Gestor-Qualidade	Gestor de Qualidade	Security
PRONEP-NF-Gestor-RH-DP	Gestor de RH/Departamento Pessoal	Security
PRONEP-NF-Gestor-Fiscal-Contabil	Gestor Fiscal-Contábil	Security
PRONEP-NF-Gestor-Juridica	Gestor da diretoria Jurídica	Security
PRONEP-NF-Gestor-Administrativa	Gestor da diretoria Administrativa	Security
PRONEP-NF-Gestor-Tecnica-SP	Gestor da diretoria Técnica em São Paulo	Security
PRONEP-NF-Gestor-Tecnica-RJES	Gestor da diretoria Técnica em Rio de Janeiro e Espírito Santo	Security

Para cada grupo:

Entra ID → **Groups** → **+ New group**

Group type: **Security**

Group name: nome literal da tabela

Membership type: **Assigned** (manual)

Adicionar os membros iniciais

Create

Após criado, abrir o grupo e **anotar o Object ID (GUID)** — você vai precisar dele

4.7 Mapear GUIDs de grupos no código

O backend tem uma tabela hardcoded que mapeia GUID → perfil. É necessário substituir os GUIDs pelos da sua nova implantação.

Arquivo: `api/MeusGrupos/index.js`, próximo à linha 24.

Substituir o objeto `GROUP_TO_ROLE` pelos GUIDs reais da sua tenant:

```
const GROUP_TO_ROLE = {
  '<GUID-PRONEP-NF-Submitter>': 'submitter',
  '<GUID-PRONEP-NF-Admin>': 'administrador',
  '<GUID-PRONEP-Financeiro-Gestao>': 'financeiro_nf',
  '<GUID-PRONEP-NF-Gestor-Suprimentos>': 'gestor_suprimentos',
  '<GUID-PRONEP-NF-Gestor-Financeira>': 'gestor_financeira',
  '<GUID-PRONEP-NF-Gestor-Tecnologia>': 'gestor_tecnologia',
  '<GUID-PRONEP-NF-Gestor-Qualidade>': 'gestor_qualidade',
  '<GUID-PRONEP-NF-Gestor-RH-DP>': 'gestor_rh_dp',
  '<GUID-PRONEP-NF-Gestor-Fiscal-Contabil>': 'gestor_fiscal_contabil',
  '<GUID-PRONEP-NF-Gestor-Juridica>': 'gestor_juridica',
  '<GUID-PRONEP-NF-Gestor-Administrativa>': 'gestor_administrativa',
}
```



```
'<GUID-PRONEP-NF-Gestor-Tecnica-SP>': 'gestor_tecnica_sp',
'<GUID-PRONEP-NF-Gestor-Tecnica-RJES>': 'gestor_tecnica_rjes'
};
```

Adicionalmente, atualizar a constante ADMIN_GROUP_ID no arquivo api/AdminLimparBase/index.js com o GUID do grupo PRONEP-NF-Admin (a função "Limpar Base" só aceita esse grupo específico).

Resultado: ao logar, o backend pesquisa os grupos do usuário e mapeia para um perfil interno. Por exemplo, alguém no grupo PRONEP-NF-Gestor-Suprimentos recebe gestor_suprimentos, e o sistema entende que esse usuário pode aprovar apenas NFs da Diretoria "Suprimentos".

5. SharePoint

A persistência do sistema é feita 100% em SharePoint Online, distribuída em 5 Listas (banco de dados) e um Document Library com 3 pastas (armazenamento dos PDFs). Esta seção descreve passo a passo a criação de cada elemento.

5.1 Criar (ou reaproveitar) o site

Acesse o SharePoint Admin Center e crie um site dedicado ou reutilize um existente. O sistema é compatível com um site novo do tipo "Team site" ou um site existente com permissões adequadas.

Recomendação: criar um site dedicado para isolamento e backup. Sugestão de URL: <https://<tenant>.sharepoint.com/sites/Aprovacao-NotasFiscaisServicos>

Após criar, anote:

- **Hostname:** parte antes do /sites/... (ex.: pronepadmin.sharepoint.com) → será SHAREPOINT_SITE_HOSTNAME
- **Path:** a partir de /sites/ (ex.: /sites/Aprovacao-NotasFiscaisServicos) → será SHAREPOINT_SITE_PATH

5.2 Lista 1 — `PRONEP-NF-NotasFiscais`

Esta é a lista central. Cada item representa uma NF lançada no sistema.

SharePoint → site → + Novo → Lista → Lista em branco

Nome: PRONEP-NF-NotasFiscais (sensível a maiúsculas)

Criar

Criar as colunas abaixo via + Adicionar coluna:

Nome (display)	Tipo (PT-BR)	Tipo (EN)	Obrigatório
Title (já vem)	Uma linha de texto	Single line of text	Sim
NumeroNF	Uma linha de texto	Single line of text	Sim
Serie	Uma linha de texto	Single line of text	Sim
CNPJFornecedor	Uma linha de texto	Single line of text	Sim
FornecedorRazao	Uma linha de texto	Single line of text	Sim
Valor	Moeda (R\$)	Currency	Sim
DataVencimento	Data e Hora	Date and Time	Sim
Unidade	Opções (SP, RJ, ES)	Choice	Sim
Diretoria	Uma linha de texto	Single line of text	Sim
Status	Opções (Lancada, AguardandoN2, Aprovada, Rejeitada)	Choice	Sim
LancadoPor	Uma linha de texto	Single line of text	Sim
LancadoEm	Uma linha de texto	Single line of text	Sim
AprovadorAtual	Uma linha de texto	Single line of text	Não
AprovadoPor	Uma linha de texto	Single line of text	Não
AprovadoEm	Uma linha de texto	Single line of text	Não

NegociadoCom	Uma linha de texto	Single line of text	Não
Descricao	Várias linhas de texto	Multiple lines of text	Não
MotivoRejeicao	Várias linhas de texto	Multiple lines of text	Não
HashSHA256	Uma linha de texto	Single line of text	Sim
UriPDF	Hiperlink	Hyperlink	Não
UriPDFAprovado	Hiperlink	Hyperlink	Não
Processado	Sim/Não	Yes/No	Não

Atenção especial: Valor precisa ser **Moeda**, UriPDF e UriPDFAprovado precisam ser **Hiperlink**. A interface clássica do SharePoint às vezes cria como Número/Texto por padrão. Após criar, abra cada coluna em Configurações e confirme o tipo. Se necessário, exclua e recrie no tipo correto.

5.3 Lista 2 — `PRONEP-NF-Fornecedores`

Cadastro de fornecedores ativos do sistema.

+ Novo → Lista → Lista em branco

Nome: PRONEP-NF-Fornecedores

Colunas:

Nome (display)	Tipo	Obrigatório
Title (já vem)	Uma linha de texto	Sim
RazaoSocial	Uma linha de texto	Sim
NomeFantasia	Uma linha de texto	Não
Documento	Uma linha de texto	Sim
TipoDocumento	Opções (CNPJ, CPF)	Sim
Categoria	Opções (Serviços, Materiais, Locação, Reembolso, Medicamentos, Outros)	Sim
DescricaoOutros	Uma linha de texto	Não
Unidade	Opções (SP, RJ, ES)	Não
Diretoria	Uma linha de texto	Não
AtendeTodas	Sim/Não	Não
Ativo	Sim/Não	Sim

A coluna Title armazena a Razão Social (mesmo conteúdo que RazaoSocial). É a chave de busca padrão na interface do SharePoint.

5.4 Lista 3 — `PRONEP-NF-Diretorias`

Matriz Unidade × Diretoria → Aprovador. Define quem aprova o quê.

+ Novo → Lista → Lista em branco

Nome: PRONEP-NF-Diretorias

Colunas:

Nome (display)	Tipo	Obrigatório
Title (já vem)	Uma linha de texto	Sim
Unidade	Opções (SP, RJ, ES)	Sim
Diretoria	Uma linha de texto	Sim
EmailAprovador	Uma linha de texto	Sim
NomeAprovador	Uma linha de texto	Não

A coluna Title deve seguir o formato Unidade|Diretoria, ex.: SP|Suprimentos, RJ|Tecnologia. Esse é o índice que o backend usa para fazer lookup quando uma NF é lançada.

Popular esta lista com a matriz completa: 3 unidades × 9 diretorias = 27 linhas. Exemplo das primeiras 3 linhas:

Title	Unidade	Diretoria	EmailAprovador	NomeAprovador
`SP	Suprimentos`	SP	Suprimentos	bruno.hioka@pronep.com.br
`SP	Tecnologia`	SP	Tecnologia	rafael.machado@pronep.com.br
`SP	Financeira`	SP	Financeira	henrico.molina@pronep.com.br

5.5 Lista 4 — `PRONEP-NF-Config`

Configuração global do sistema (multi-nível, gestor master, etc.). Apenas 1 item.

+ Novo → Lista → Lista em branco

Nome: PRONEP-NF-Config

Colunas:

Nome (display)	Tipo	Obrigatório
Title (já vem)	Uma linha de texto	Sim
ConfigJson	Várias linhas de texto	Sim

Após criar a lista, adicionar 1 único item:

- Title: global
- ConfigJson:

```
{ "multiNivel":
{ "habilitado":false,"valorLimite":0,"modoAprovador":"global","gestorMasterGlobal":"","gestoresPorDiretoria":{ } }
```

A configuração pode ser ajustada posteriormente pela tela Configurações do sistema (interface do admin).

5.6 Lista 5 — `PRONEP-NF-PushSubscriptions`

Armazena as assinaturas de Push Notification de cada dispositivo de cada usuário. Esta lista é opcional — só é necessária se você for habilitar Push Notifications.

+ Novo → Lista → Lista em branco

Nome: PRONEP-NF-PushSubscriptions

Colunas:

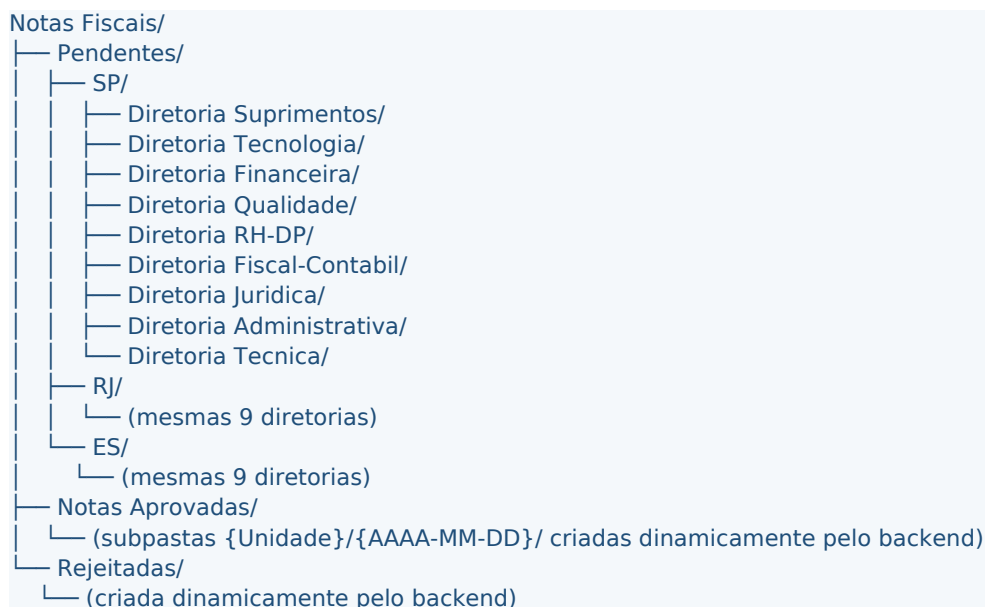
Nome (display)	Tipo	Obrigatório
Title (já vem)	Uma linha de texto	Sim
Endpoint	Várias linhas de texto	Sim
P256DH	Uma linha de texto	Sim
Auth	Uma linha de texto	Sim
UserAgent	Uma linha de texto	Não

Endpoint precisa ser **Várias linhas de texto** porque URLs de push service do Firebase/Apple/Mozilla podem passar de 250 caracteres.

5.7 Pastas no Document Library

O Document Library padrão chamado **Documents** (ou **Documentos**) recebe o upload dos PDFs.

Estrutura a criar manualmente (uma única vez):



A pasta Notas Aprovadas pode ser criada vazia — o backend criará subpastas por unidade e data automaticamente. O mesmo vale para Rejeitadas.

5.8 Permissões do site para a App Registration

O backend (Azure Functions) faz chamadas Graph autenticadas com o Client ID e Client Secret da App Registration. Para que essas chamadas tenham acesso à site específico, o admin do tenant precisa garantir uma das duas abordagens:

Abordagem A — Sites.ReadWrite.All (mais simples, mais permissivo): já concedido na seção 4.4. Não precisa fazer mais nada. A App Reg tem acesso a TODOS os sites do tenant.

Abordagem B — Sites.Selected (mais restrito, recomendado em ambientes com governança rígida): ao invés de conceder Sites.ReadWrite.All, concede-se apenas Sites.Selected na App Reg e depois libera-se via Graph apenas o site específico. Requer um POST adicional no Graph autorizando a App Reg no site. Documentação: docs.microsoft.com/graph/sites-set-permissions

Para a Pronep, a Abordagem A é a usada atualmente. Mais simples, e como a Pronep tem governança interna de App Registrations, é aceitável.

6. Azure Static Web App

Esta seção provisiona o Azure Static Web App, que é o "container" onde o frontend e as Functions vão rodar.

6.1 Criar o recurso

Portal Azure → buscar **Static Web Apps** → **+ Create**

Preencher:

- **Subscription:** a do projeto
- **Resource Group:** criar novo (rg-aprovacao-nf) ou reaproveitar
- **Name:** aprovacao-nf-pronep (será o subdomínio: aprovacao-nf-pronep.azurestaticapps.net)
- **Plan type:** **Standard** (necessário para Teams SSO e configuração customizada de auth)

- **Region:** East US 2 ou West Europe (regiões com melhor cobertura de Functions Node 22)
- **Source:** **GitHub**
- Autenticar com a conta GitHub
- Selecionar a Organização e o **Repository** onde o código está
- Branch: main
- **Build presets:** Custom
- **App location:** /wwwroot
- **Api location:** /api
- **Output location:** (deixar vazio)

Review + Create → Create

Aguardar o provisionamento (~2 minutos). Anotar a URL pública: `https://<nome>-<hash>.<region>.azurestaticapps.net`

O Azure já cria automaticamente um workflow GitHub Actions no repositório (arquivo `.github/workflows/azure-static-web-apps-<algo>.yaml`). O primeiro deploy é disparado em seguida.

6.2 Configurar Redirect URI na App Registration

Voltar ao Entra ID → App Registration criada na seção 4.1 → **Authentication**.

+ Add a platform → Web

Redirect URI: `https://<host-do-swa>/.auth/login/aad/callback`

- Exemplo: `https://aprovacao-nf-pronep.azurestaticapps.net/.auth/login/aad/callback`

Configure

Confirme que a opção **ID tokens** está marcada (já feita na seção 4.3)

6.3 Configurar App Settings (variáveis de ambiente)

Portal Azure → seu Static Web App → menu lateral **Configuration** (ou Configuração) → guia **Application settings** → **+ Add** uma por uma.

Variáveis obrigatórias para o sistema funcionar:

Nome	Valor	Origem
AAD_TENANT_ID	GUID do tenant	Entra ID → Overview
AAD_CLIENT_ID	Client ID da App Reg	seção 4.1
AAD_CLIENT_SECRET	valor copiado	seção 4.2
SHAREPOINT_SITE_HOSTNAME	hostname do tenant	ex.: pronepadmin.sharepoint.com
SHAREPOINT_SITE_PATH	path com /sites/...	ex.: /sites/Aprovacao-NotasFiscaisServicos
EMAIL_FROM_ADDRESS	conta de envio	ex.: datanalytics@pronep.com.br

Variáveis opcionais (mas recomendadas):

Nome	Valor	Quando preencher
APP_ID_URI	<code>api://<host>/<client_id></code>	Se Teams SSO ativo
LINK_APROVACAO_SECRET	string aleatória (32+ chars)	Para botões Aprovar/Rejeitar no e-mail
VAPID_PUBLIC_KEY	gerada via Python	Se Push Notifications ativo
VAPID_PRIVATE_KEY	gerada via Python	Se Push Notifications ativo
VAPID_SUBJECT	<code>mailto:<email></code>	Push Notifications
TEAMS_APP_ID	GUID do manifest	Se Teams Activities ativos

Após adicionar todas, clicar **Save** no topo. Em seguida, **Overview** → **Restart** para garantir que as Functions peguem as novas envs.

6.4 Verificar que a autenticação está ativa

Acessar a URL pública do SWA. O navegador deve redirecionar para login.microsoftonline.com. Após login com uma conta do tenant, redireciona de volta para o app.

Se aparecer "Sem acesso" (status 403): o usuário precisa estar em algum dos 13 grupos. Adicionar o admin testador no grupo PRONEP-NF-Admin.

7. GitHub — Repositório e CI/CD

7.1 Estrutura do workflow

O Azure já criou automaticamente o arquivo .github/workflows/azure-static-web-apps-<hash>.yaml na primeira conexão. O conteúdo final desejado é:

```
name: Azure Static Web Apps CI/CD

on:
  push:
    branches: [main]
  pull_request:
    types: [opened, synchronize, reopened, closed]
    branches: [main]

jobs:
  build_and_deploy_job:
    if: github.event_name == 'push' || (github.event_name == 'pull_request' && github.event.action != 'closed')
    runs-on: ubuntu-latest
    name: Build And Deploy
    steps:
      - uses: actions/checkout@v4
        with:
          submodules: true
          lfs: false
      - name: Build And Deploy
        id: builddeploy
        uses: Azure/static-web-apps-deploy@latest
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: "upload"
          app_location: "/wwwroot"
          api_location: "/api"
          output_location: ""

  close_pull_request_job:
    if: github.event_name == 'pull_request' && github.event.action == 'closed'
    runs-on: ubuntu-latest
    name: Close Pull Request Job
    steps:
      - name: Close Pull Request
        id: closepullrequest
        uses: Azure/static-web-apps-deploy@latest
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }
          action: "close"
```

Pontos importantes:

- A versão da action deve ser `@latest`. Versões antigas (@v1) usavam SHAs que foram movidos e quebram o build com erro de download.
- app_location: "/wwwroot" — onde está o frontend
- api_location: "/api" — onde estão as Functions
- output_location: "" — vazio, não há etapa de build

7.2 Secret do Actions

O Azure já cria automaticamente o secret AZURE_STATIC_WEB_APPS_API_TOKEN no repositório GitHub quando você conectou o SWA pela primeira vez. Verificar em:

GitHub → Repositório → **Settings** → **Secrets and variables** → **Actions** → deve aparecer o secret listado.

Se algum dia o token precisar ser renovado: Portal Azure → SWA → **Overview** → **Manage deployment token** → reset → copiar novo → atualizar no GitHub.

7.3 Disparar o primeiro deploy

Após qualquer push na branch main, o workflow é executado automaticamente. Para acompanhar:

GitHub → Repositório → **Actions** → ver o último run com nome do commit.

Quando o status ficar verde (ícone de check), o deploy foi bem-sucedido. O sistema já está acessível na URL pública.

Em caso de falha, abrir o run e ler os logs. As causas mais comuns são: secret expirado, erro de sintaxe em algum index.js da API, ou falha de instalação de dependências (verificar api/package.json).

8. Microsoft Teams App

Se a empresa quer permitir que os aprovadores recebam notificações 1:1 no Teams e abram o sistema como Personal Tab dentro do próprio Teams, é necessário publicar um Teams App.

8.1 Estrutura do pacote

O pacote Teams App é um arquivo .zip contendo:

```
aprovacao-nf-teams.zip
├─ manifest.json  (descriptor JSON do app)
├─ color.png      (ícone colorido 192x192)
└─ outline.png    (ícone outline 32x32)
```

Estes arquivos já existem no repositório em teams-app/. Apenas o manifest.json precisa ser ajustado para o novo tenant.

8.2 Ajustar o manifest

Editar teams-app/manifest.json. Substituir 4 valores:

```
{
  "id": "<GERAR-NOVO-GUID>",
  ...
  "developer": {
    "name": "Pronep Life Care",
    "websiteUrl": "https://www.pronep.com.br",
    "privacyUrl": "https://www.pronep.com.br/privacidade",
  }
}
```

```
"termsOfUseUrl": "https://www.pronep.com.br/termos"
},
...
"staticTabs": [
  {
    "entityId": "aprovacao-nf-home",
    "name": "Aprovacao NF",
    "contentUrl": "https://<HOST-DO-SWA>/",
    "websiteUrl": "https://<HOST-DO-SWA>/",
    "scopes": ["personal"]
  }
],
...
"validDomains": ["<HOST-DO-SWA>"],
"webApplicationInfo": {
  "id": "<CLIENT-ID-DA-APP-REG>",
  "resource": "api://<HOST-DO-SWA>/<CLIENT-ID-DA-APP-REG>"
}
}
```

Onde:

- <GERAR-NOVO-GUID> — qualquer GUID válido (use <https://www.uuidgenerator.net> ou PowerShell [guid]::NewGuid()). É o externalId do Teams App.
- <HOST-DO-SWA> — domínio do Static Web App, sem https://. Exemplo: aprovacao-nf-pronep.azurestaticapps.net
- <CLIENT-ID-DA-APP-REG> — Client ID da App Registration criada na seção 4.1

Salvar.

8.3 Empacotar e fazer upload

Em PowerShell, dentro da pasta teams-app/:

```
Compress-Archive -Path .\manifest.json,.\color.png,.\outline.png `
-DestinationPath aprovacao-nf-teams.zip -Force
```

Em seguida:

Microsoft Teams Admin Center → **Teams apps** → **Manage apps**

+ **Upload new app** → **Upload**

Selecionar o aprovacao-nf-teams.zip

Aguardar processamento (~30 segundos)

Localizar o app na lista → garantir que esteja com status **Allowed**

8.4 Política de instalação

Para que os aprovadores recebam Activity Notifications, o Teams App precisa estar **instalado** no Teams pessoal de cada um. O sistema instala automaticamente via Graph (permissão TeamsAppInstallation.ReadWriteForUser.All), mas a política do tenant precisa permitir.

Teams Admin Center → **Setup policies** → política aplicável aos usuários → seção **Installed apps** → + **Add apps** → adicionar o "Aprovacao NF" como app pré-instalado, OU garantir que a política não bloqueia instalações via Graph.

8.5 Atualizar o manifest no futuro

Toda vez que mudar algo no manifest (URL, ícone, nova activity, etc.), incrementar version (ex.: 1.0.1 → 1.0.2), gerar novo ZIP e fazer **Update** no Teams Admin Center.

9. Notificações Push (opcional)

O sistema usa Web Push Notifications via VAPID para enviar alertas nativos no celular e desktop quando uma NF cai na fila do aprovador. É um **canal adicional** ao e-mail e Teams — não substitui nenhum dos dois.

Se a empresa não quiser usar push notifications, esta seção pode ser pulada e o sistema funcionará apenas com e-mail + Teams.

9.1 Gerar VAPID keys

VAPID (Voluntary Application Server Identification) é um padrão que usa um par de chaves criptográficas para autenticar o servidor que envia push. As keys são geradas **uma única vez**, e usadas para sempre.

Em um terminal com Python:

```
python3 -c "  
from cryptography.hazmat.primitives.asymmetric import ec  
from cryptography.hazmat.primitives import serialization  
import base64  
priv = ec.generate_private_key(ec.SECP256R1())  
pub = priv.public_key()  
priv_bytes = priv.private_numbers().private_value.to_bytes(32, 'big')  
pub_bytes = pub.public_bytes(  
    encoding=serialization.Encoding.X962,  
    format=serialization.PublicFormat.UncompressedPoint)  
print('VAPID_PUBLIC_KEY:', base64.urlsafe_b64encode(pub_bytes).rstrip(b'=').decode())  
print('VAPID_PRIVATE_KEY:', base64.urlsafe_b64encode(priv_bytes).rstrip(b'=').decode())  
"
```

Saída esperada (exemplo):

```
VAPID_PUBLIC_KEY : BJP9M_UiRSmns5A7cVrCCvvMH1QVfSUza15RC...  
VAPID_PRIVATE_KEY : G5wTtN9bNvSffyR-1008r4vAp6gJgi6Yx9YfKIhwsHY
```

Importante: a VAPID_PRIVATE_KEY é secreta. Trate como uma senha. Guarde em cofre. Nunca commit no Git.

9.2 Adicionar as keys no Azure SWA

Portal Azure → SWA → **Configuration** → **+ Add** três variáveis:

Nome	Valor
VAPID_PUBLIC_KEY	a pública gerada
VAPID_PRIVATE_KEY	a privada gerada
VAPID_SUBJECT	mailto:datanalytics@pronep.com.br

Save → **Restart** do SWA.

9.3 Criar a lista PRONEP-NF-PushSubscriptions

Já descrita na seção 5.6. Se ainda não foi criada, crie agora.

9.4 Ativar pelo lado do usuário

Cada usuário precisa ativar individualmente em cada dispositivo:

Acessa o sistema → menu **Configurações**

Card **Notificações no celular / desktop** → botão ☐ **Ativar notificações**

Browser pede permissão → **Permitir**

Status muda para "Notificações ativadas neste dispositivo"

Em **iPhone**, é obrigatório ter primeiro adicionado o app à Tela de Início (PWA instalado). iOS Safari só permite push em PWAs instalados.

Parte III — Referência técnica

10. Estrutura do projeto

C:\Pronep\Aprovacao_NF\

- README / docs —
 - LEIA-ME.md
 - GUIA_ENTRA_ID.md
 - GUIA_AZURE_SWA.md
 - GUIA_SHAREPOINT.md
 - SETUP_TIPOS_COLUNA_SHAREPOINT.md
 - SETUP_TEAMS_SSO.md
 - SETUP_TEAMS_ACTIVITY.md
 - SETUP_PUSH_NOTIFICATIONS.md
 - ROADMAP_SPRINT_4_FECHAMENTO.md
- .github/workflows/
 - azure-static-web-apps-<hash>.yml ← CI/CD GitHub Actions
- wwwroot/ — Frontend SPA + PWA —
 - index.html ← arquivo principal (~5000 linhas, HTML + CSS + JS)
 - staticwebapp.config.json ← rotas, Easy Auth, CSP
 - manifest.webmanifest ← PWA manifest
 - sw.js ← Service Worker (PWA + cache offline + push)
 - offline.html ← fallback offline
 - pronep-logo.png
 - favicon.svg / favicon-256.png / apple-touch-icon.png / icon-192.png / icon-512.png
 - teams-login.html ← popup auth Teams (fallback)
 - teams-auth-callback.html ← callback do popup
 - sem-acesso.html ← 403
 - vendor/
 - chart.umd.min.js (Chart.js)
 - teams-js.min.js (Microsoft Teams JS SDK)
 - xlsx.full.min.js (SheetJS)
- api/ — Azure Functions (Node.js v22) —
 - host.json
 - package.json
 - shared/
 - auth.js ← Easy Auth + Teams SSO validation
 - email.js ← orquestrador de notificações (email + Teams + push)
 - teamsActivity.js ← sendActivityNotification
 - pushNotif.js ← Web Push (VAPID)
 - notificar.js ← re-export de email.js
 - Hello/ ← GET /api/Hello (health check)
 - MeusGrupos/ ← GET /api/MeusGrupos (mapeia grupos → roles)
 - ListarNotas/ ← GET /api/ListarNotas (lista filtrada por escopo)
 - PostNota/ ← POST /api/PostNota (lança NF)
 - AprovarNota/ ← POST /api/AprovarNota (aprova + watermark)
 - RejeitarNota/ ← POST /api/RejeitarNota (rejeita)
 - AprovacaoViaLink/ ← GET /api/AprovacaoViaLink (botão do email)
 - ListarFornecedores/ ← GET /api/ListarFornecedores
 - AdicionarFornecedor/ ← POST /api/AdicionarFornecedor
 - EditarFornecedor/ ← PATCH /api/EditarFornecedor
 - ListarDiretorias/ ← GET /api/ListarDiretorias
 - ListarGestoresFinanceiro/ ← GET /api/ListarGestoresFinanceiro
 - ConsultarCNPJ/ ← GET /api/ConsultarCNPJ (Brasilapi proxy)
 - AbrirPdfDaNota/ ← GET /api/AbrirPdfDaNota (redirect 302 pro PDF)

└─ MarcarProcessado/	← POST /api/MarcarProcessado
└─ ConfigGet/	← GET /api/ConfigGet
└─ ConfigUpdate/	← POST /api/ConfigUpdate
└─ PushPublicKey/	← GET /api/PushPublicKey
└─ PushSubscribe/	← POST /api/PushSubscribe
└─ PushUnsubscribe/	← POST /api/PushUnsubscribe
└─ AdminLimparBase/	← POST /api/AdminLimparBase (reset admin)
└─ EnviarNotificacao/	← POST /api/EnviarNotificacao (utilitário)
└─ teams-app/	└─ Pacote Teams App
└─ manifest.json	
└─ color.png	
└─ outline.png	

11. Catálogo de Azure Functions

O backend tem 22 endpoints HTTP. Cada Function vive numa pasta dentro de api/. Cada pasta contém:

- function.json — descritor que define o trigger (HTTP, métodos aceitos, nível de auth)
- index.js — código JavaScript da Function (módulo CommonJS)

Convenções gerais:

- Toda Function é **HTTP-triggered**, com authLevel: anonymous (a validação real de auth acontece dentro do código via shared/auth.js)
- Toda Function retorna JSON
- Erros retornam objetos { error: "mensagem", diag: { step: "..." } } para diagnóstico
- Todas usam getUser(req) de shared/auth.js para identificar o usuário autenticado (exceto Hello e endpoints públicos)

11.1 Endpoints de autenticação e perfil

GET /api/Hello — health check. Verifica se as variáveis de ambiente principais estão configuradas. Não exige autenticação. Útil pra confirmar deploy.

GET /api/MeusGrupos — retorna o perfil do usuário autenticado. Consulta Graph transitiveMemberOf do user, cruza com a tabela GROUP_TO_ROLE hardcoded, retorna o perfil aplicável (admin, financeiro, gestor_X, submitter). O frontend chama esta Function logo no boot para decidir quais telas mostrar.

11.2 Endpoints de Notas Fiscais (núcleo)

POST /api/PostNota — lança uma nova NF. Body: { fornecedorCNPJ, fornecedorRazao, numero, serie, valor, vencimento, unidade, diretoria, negociadoCom, descricao, fileBase64, fileName }. Fluxo: valida campos, calcula hash SHA-256 do PDF, consulta lista por duplicidade, resolve aprovador via Diretorias, sobe PDF em Notas Fiscais/Pendentes/{Unidade}/Diretoria {Diretoria}/, cria item na lista NotasFiscais com Status=Lancada, dispara notificar('lancada', ...). Retorna { ok, id, urlPDF, aprovador }. Erros: 400 (validação), 409 (duplicidade), 500 (erro Graph).

GET /api/ListarNotas — lista NFs visíveis ao usuário. RBAC server-side: admin/financeiro veem tudo; gestor vê apenas onde AprovadorAtual = seu email; submitter vê apenas onde LancadoPor = seu email. Aceita querystring ?status=Lancada|Aprovada|Rejeitada para filtro. Retorna { notas: [...] }.

POST /api/AprovarNota — aprova uma NF. Body: { id }. Valida que o solicitante é o AprovadorAtual ou tem perfil de override (admin). Consulta a config multiNivel: se valor da NF excede o limite e ainda não está em "AguardandoN2", atualiza Status=AguardandoN2 e o AprovadorAtual

para o gestor master (sem mover o PDF). Caso contrário: baixa o PDF, aplica watermark "APROVADO", move para Notas Aprovadas/{Unidade}/{AAAA-MM-DD}/, atualiza Status=Aprovada, dispara notificar('aprovada', ...).

``POST /api/RejeitarNota`` — rejeita uma NF. Body: { id, motivo, observacao }. Aplica watermark "REJEITADA", move PDF para Notas Fiscais/Rejeitadas/, Status=Rejeitada, dispara notificar('rejeitada', ...).

``GET /api/AprovacaoViaLink?token=...`` — recebe JWT assinado pelo backend e executa aprovação/rejeição sem exigir login. Usado pelos botões inline no e-mail. Token é assinado com LINK_APROVACAO_SECRET, validade 7 dias.

``GET /api/AbrirPdfDaNota?id=...`` — resolve a URL do PDF no Drive e retorna redirect 302 para preview do SharePoint. Verifica Status para determinar se busca em Pendentes, Aprovadas ou Rejeitadas. Inclui salvaguarda contra "PDF errado" (não usa fallback "mais recente" — exige match exato pelo número da NF).

``POST /api/MarcarProcessado`` — marca/desmarca o campo Processado de uma NF. Usado pelo financeiro após integrar a NF ao sistema fiscal.

11.3 Endpoints de Fornecedores

``GET /api/ListarFornecedores`` — retorna todos os fornecedores cadastrados. Suporta paginação Graph via @odata.nextLink. Resolve internal names dinamicamente.

``POST /api/AdicionarFornecedor`` — cria fornecedor. Body: { razao, nomeFantasia, documento, tipoDocumento, unidade, diretoria, categoria, atendeTodas, ativo }.

``PATCH /api/EditarFornecedor`` — atualiza fornecedor. Body: { id, ...campos }.

``GET /api/ConsultarCNPJ?cnpj=...`` — proxy para Brasilapi, retorna dados públicos do CNPJ (razão, nome fantasia, status na Receita).

11.4 Endpoints de Configuração

``GET /api/ListarDiretorias`` — retorna a matriz Unidade × Diretoria → Aprovador. Usado pelo frontend para popular dropdowns.

``GET /api/ListarGestoresFinanceiro`` — retorna membros do grupo PRONEP-Financeiro-Gestao via Graph. Usado pelo combobox "Negocieie com" no lançamento de NF.

``GET /api/ConfigGet`` — retorna o JSON da config global (multi-nível, etc.).

``POST /api/ConfigUpdate`` — atualiza a config. Restrito a admin.

11.5 Endpoints de Push Notifications

``GET /api/PushPublicKey`` — retorna { publicKey: VAPID_PUBLIC_KEY } para o frontend usar no pushManager.subscribe().

``POST /api/PushSubscribe`` — recebe a subscription do navegador e salva no SharePoint. Body: { subscription: { endpoint, keys }, userAgent }.

``POST /api/PushUnsubscribe`` — remove a subscription. Body: { endpoint }.

11.6 Endpoints administrativos

``POST /api/AdminLimparBase`` — apaga todos os itens da lista NotasFiscais e remove PDFs das pastas Pendentes e Rejeitadas. Protegido por tripla validação: (a) usuário precisa ter role admin, (b) body precisa conter { confirmacao: "LIMPAR" }, (c) método POST. Pasta Notas Aprovadas é preservada por padrão (histórico legacy).

12. Módulos compartilhados (`api/shared/`)

`auth.js`

Resolve a identidade do chamador a partir de dois caminhos possíveis:

Easy Auth (browser): header x-ms-client-principal injetado pelo SWA. Decodifica o JSON Base64, extrai o oid (Object ID) dos claims, valida o formato GUID.

Teams SSO (iframe do Teams): header customizado X-Teams-Token (porque o SWA Easy Auth sobrescreve o header Authorization). Valida o JWT contra JWKS público da Microsoft, confere audience e issuer.

Função exportada: `async getUser(req) → retorna { email, name, oid, source } ou null.`

`email.js`

Orquestra notificações em 3 canais quando um evento acontece. Função exportada: `async notificar(evento, destinatarios, dados)`. Eventos suportados: lancada, aprovada, rejeitada.

Sub-funções:

- `enviarEmail()` — usa Graph `users/{email}/sendMail` com a conta `EMAIL_FROM_ADDRESS`. Inclui botões Aprovar/Rejeitar assinados como JWT no caso de evento lancada.
- `enviarTeamsAtividade()` — chama `teamsActivity.js`.
- `enviarPushParaDestinatarios()` — chama `pushNotif.js`.

`teamsActivity.js`

Envia notificação 1:1 no Microsoft Teams via Graph `sendActivityNotification`.

Sub-funções:

- `getCatalogAppId()` — descobre o `catalogAppId` do Teams App publicado, filtrando por `externalId` (o ID do manifest).
- `garantirAppInstalada()` — antes de enviar a notificação, garante que o Teams App está instalado para o usuário (instala via Graph se necessário).
- `enviarTeamsAtividade(evento, dados, email)` — envia a notif com template e parâmetros.

Templates dos `activityTypes` são definidos no `teams-app/manifest.json`.

`pushNotif.js`

Envia Web Push notifications. Função principal: `enviarPushPraEmail(client, siteId, email, payload)`.

Sub-funções:

- `configurarWebPush()` — inicializa a lib `web-push` com as VAPID keys (no-op se variáveis não configuradas).
- `listarSubscriptionsPorEmail()` — consulta a lista `PRONEP-NF-PushSubscriptions` pelas `subscriptions` do email.
- `enviarPushPraEmail()` — envia push para todas as `subscriptions` do user; `subscriptions` inválidas (HTTP 410 do push service) são removidas automaticamente.

`notificar.js`

Apenas re-exporta `notificar` de `email.js` para compatibilidade com versões antigas do código que importavam de `./notificar`.

13. Frontend — SPA

O frontend é uma SPA vanilla (HTML/CSS/JS) em um único arquivo: `wwwroot/index.html`. Sem framework, sem build step. Roda diretamente no navegador após o SWA servir o arquivo estático.

Camadas internas do `index.html`

`<head>`: meta tags (viewport, theme-color, PWA), link para manifest.webmanifest, scripts dos vendors (Chart.js, Teams JS SDK, SheetJS).

`<style>`: CSS inline (~700 linhas) — design tokens, layout grid (sidebar + content), responsividade @media.

`<body>`: marcação HTML da tela de login + shell do app (topbar, sidebar, content area, modal, toasts, footer mobile).

`<script>` (~3500 linhas): toda a lógica do app, organizada em blocos lógicos (auth, RBAC, views, formulários, helpers de data/moeda, integrações Teams/PWA/Push).

Views (telas) implementadas

View	Descrição	Acessível por
dashboard	KPIs + gráficos (3 gráficos: Diretoria×Unidade, Distribuição por Unidade, Fornecedor×Unidade)	admin, financeiro
fila-aprovacao	Lista de NFs pendentes filtradas por escopo do usuário	todos com escopo
aprovadas	Histórico de NFs aprovadas, com filtros e checkbox Processado	admin, financeiro
rejeitadas	NFs rejeitadas com motivo	admin, financeiro
rejeitadas-minhas	Apenas as próprias rejeitadas (submitter)	submitter
minhas-nfs	NFs lançadas pelo próprio user	todos
nova-nf	Formulário de lançamento de NF	submitter (e qualquer com escopo)
fornecedores	CRUD de fornecedores + importação XLSX	admin, financeiro
mapa-aprovadores	Visualização da matriz Unidade × Diretoria → Aprovador	todos
auditoria	Log de eventos do sistema	admin
configuracoes	Multi-nível, gestores master, atalho pro Entra ID, push, zona de risco (limpar base)	admin

Mobile e PWA

Em telas ≤768px (mobile):

- O sidebar vira um **drawer** abrigado atrás de um botão hamburger no topbar
- A topbar fica compacta, e info do usuário desce pro rodapé
- As listas (Fila, Aprovadas, Rejeitadas) viram **cards estilo Planner** — cada NF é um card expansível com expand-on-click
- O Dashboard rearranja para cards empilhados verticalmente
- Filtros viram grid de 2 colunas (ou stack vertical em casos específicos)

A versão desktop é preservada (CSS controla com @media).

Service Worker (`sw.js`)

Estratégia: **cache-first para o shell** (HTML, ícones, vendors), **network-first para /api/**** (sempre busca dados frescos do backend).

Pontos cuidadosos:

- **NUNCA** intercepta /api/*, /.auth/* ou URLs externas (Microsoft, Graph, Brasilapi) — apenas pass-through
- **NUNCA** intercepta POST/PATCH/DELETE — apenas GET de navegação
- Listener push — recebe payload do servidor e mostra notificação nativa
- Listener notificationclick — abre o app na URL certa quando user clica na notif

Comunicação com o backend

Todas as chamadas usam `fetch('/api/<nome>')` com `credentials: 'include'`. O SWA injeta o cookie de Easy Auth automaticamente, e o backend tem acesso ao header `x-ms-client-principal`.

Em iframe Teams, o frontend obtém o token via `microsoftTeams.authentication.getAuthToken()` e adiciona no header customizado `X-Teams-Token` em **todas** as chamadas `fetch` (interceptor).

14. Variáveis de ambiente — referência completa

Todas as variáveis abaixo são configuradas no **Azure SWA** → **Configuration** → **Application settings**. Restart do SWA é necessário após adicionar ou alterar qualquer uma.

14.1 Obrigatórias

Variável	Tipo	Origem	O que faz
AAD_TENANT_ID	GUID	Entra ID → Overview	Tenant ID do Azure AD
AAD_CLIENT_ID	GUID	App Registration → Overview	Client ID da App Reg
AAD_CLIENT_SECRET	string (secreta)	App Reg → Certificates & secrets	Client secret para auth Application
SHAREPOINT_SITE_HOSTNAME	string	URL do tenant	Hostname do SP (ex.: pronepadmin.sharepoint.com)
SHAREPOINT_SITE_PATH	string	URL do site	Path do site (ex.: /sites/Aprovacao-NotasFiscaisServicos)
EMAIL_FROM_ADDRESS	email	escolha admin	Remetente dos emails (ex.: datanalytics@pronep.com.br)

14.2 Opcionais (com defaults internos ou só ativam features específicas)

Variável	Quando preencher	O que faz
APP_ID_URI	Se Teams SSO ativo	Application ID URI (formato <code>api://<host>/<client_id></code>)
LINK_APROVACAO_SECRET	Se usar botões Aprovar/Rejeitar no e-mail	Secret para assinar JWT dos links (32+ chars aleatórios)
VAPID_PUBLIC_KEY	Se Push ativo	Chave pública VAPID (compartilhada com frontend)
VAPID_PRIVATE_KEY	Se Push ativo	Chave privada VAPID (secreta)
VAPID_SUBJECT	Se Push ativo	<code>mailto:<email_contato></code> para padrão VAPID
TEAMS_APP_ID	Se Teams Activity Notifications ativas	<code>externalId</code> do Teams App (do manifest)

14.3 Geradores rápidos

LINK_APROVACAO_SECRET (string aleatória):

```
# Linux/Mac/WSL
```

```
openssl rand -base64 48 | tr -d '\n'
```

```
# PowerShell
```

```
[Convert]::ToBase64String((1..48 | %{Get-Random -Maximum 256}))
```

VAPID keys: ver seção 9.1.

Parte IV — Operação

15. Fluxos do sistema

15.1 Fluxo de lançamento de NF

Solicitante abre o sistema (browser, app desktop ou Teams) → menu **Lançamento NF**.

Digita parte da razão social ou CNPJ → autocomplete filtra a lista de fornecedores.

Ao selecionar o fornecedor:

- Categoria, Diretoria e Unidade são auto-preenchidos do cadastro
- Se o fornecedor atende as 3 unidades (AtendeTodas=true), 3 radios de Unidade (SP/RJ/ES) ficam habilitados; senão, apenas a Unidade do cadastro fica pré-selecionada e os outros radios desabilitados
- Se Categoria for "Outros", aparece o campo "Descrição do serviço" (obrigatório, UPPERCASE)

Preenche número da NF, série, valor (com máscara monetária BR), vencimento.

Se vencimento < D+5 dias úteis (calculado considerando feriados nacionais Brasilapi), aparece dropdown obrigatório "Negocie com" listando os gestores financeiros (do grupo PRONEP-Financeiro-Gestao). Sem essa seleção, o lançamento é bloqueado.

Faz upload do PDF (máx 6 MB) via drag-and-drop ou clique.

Clica **Enviar para SharePoint**.

Backend processa: hash, duplicidade, validações, upload, criação do item, notificações.

Modal de confirmação mostra o ID da NF e o aprovador roteado.

15.2 Fluxo de aprovação (caminho rápido — e-mail)

Aprovador recebe e-mail com botões Aprovar / Rejeitar.

Clica em **Aprovar**.

Backend valida o JWT do link, executa AprovarNota, aplica watermark e move o PDF.

Aprovador é redirecionado a uma página de confirmação visual.

Solicitante recebe notificação de aprovação (email + Teams + push).

15.3 Fluxo de aprovação (caminho UI)

Aprovador entra no sistema → menu **Fila de Aprovação**.

Lista mostra apenas as NFs do seu escopo (RBAC server-side).

Clica numa NF → modal abre com detalhes + preview do PDF.

Clica em **Aprovar** ou **Rejeitar** (no caso de rejeitar, preencher motivo).

Mesmo fluxo do backend (com ou sem 2º nível dependendo do valor + config).

15.4 Fluxo multi-nível (NFs acima do limite)

Admin habilita multi-nível em **Configurações** → marca "Habilitar aprovação multi-nível" → define o valor limite (ex.: R\$ 30.000) → define gestor master global ou por diretoria.

Quando uma NF é lançada com valor acima do limite, ela é roteada normalmente para o gestor da diretoria (1º nível).

Gestor da diretoria aprova. Backend detecta multi-nível ativo + valor > limite + Status=Lancada → atualiza Status=AguardandoN2 e o AprovadorAtual para o gestor master.

Sem mover o PDF nem aplicar watermark ainda. A NF aparece no topo da Fila do gestor master.

Gestor master aprova. Backend aplica watermark e move PDF normalmente.

15.5 Fluxo de rejeição e re-lançamento

Aprovador rejeita uma NF informando motivo.

PDF é movido para Notas Fiscais/Rejeitadas/ com watermark "REJEITADA".

Status=Rejeitada.

Solicitante recebe notificação.

Solicitante pode corrigir a NF (refazer upload) e re-lançar. O sistema **não bloqueia por duplicidade** quando a NF anterior está com Status=Rejeitada — esse comportamento foi ajustado especificamente para permitir correção.

16. Perfis de usuário

Perfil	Pode ver	Pode fazer
Administrador (grupo PRONEP-NF-Admin)	Tudo do sistema, todas as unidades, todas as diretorias	Tudo, incluindo Configurações, Auditoria, Limpar Base
Financeiro (grupo PRONEP-Financeiro-Gestao)	Dashboard, Fila, Aprovadas, Rejeitadas, Fornecedores (tudo)	Aprovar como 2º nível (se multi-nível ativo), marcar Processado
Gestor (grupos PRONEP-NF-Gestor-*)	NFs apenas da sua diretoria	Aprovar/Rejeitar NFs da sua diretoria
Solicitante (grupo PRONEP-NF-Submitter)	NFs próprias	Lançar NF, ver minhas-nfs, ver minhas-rejeitadas

Quando um usuário tem múltiplos grupos, o sistema aplica o **perfil de mais alta privilégio** (admin > financeiro > gestor > submitter).

17. Notificações

O sistema envia notificações em 3 canais paralelos. Cada um pode ser ativado/desativado por usuário.

E-mail

- Sempre enviado (canal padrão obrigatório)
- HTML branded com identidade Pronep
- Botões inline Aprovar/Rejeitar (JWT assinado, validade 7 dias)
- Remetente: EMAIL_FROM_ADDRESS

Microsoft Teams

- Notificação 1:1 via Activity Notification (sino do Teams)
- Usuário precisa ter o Teams App instalado (auto-instalação na primeira notif)
- Política do tenant precisa permitir Custom Apps

Push Notifications

- Canal opcional. Usuário ativa em **Configurações** → **Notificações no celular/desktop**
- Funciona em Android (Chrome), Windows/Mac (Chrome/Edge) e iPhone (PWA instalado)
- Cada dispositivo gera uma subscription independente. Usuário pode ativar em vários.

Eventos que disparam notificação

Evento	Para quem
NF lançada	Aprovador resolvido pela matriz
NF aprovada	Solicitante + financeiro
NF rejeitada	Solicitante
2º nível pendente	Gestor master (apenas se multi-nível ativo)

18. Backups e auditoria

Backup das listas SharePoint

Recomendação operacional: exportar semanalmente para XLSX as listas PRONEP-NF-Fornecedores e PRONEP-NF-Diretorias (manual ou via Power Automate). Lista PRONEP-NF-NotasFiscais é cumulativa, mas vale exportar mensalmente como snapshot.

Os PDFs no Document Library têm versionamento nativo do SharePoint.

Auditoria

Hoje o sistema mostra uma tela **Auditoria** com dados mockados (personas hardcoded). Para auditoria real em produção, recomenda-se:

Habilitar **Application Insights** no Static Web App

Ativar Log Analytics workspace

Cada Function já loga via context.log e context.log.error — esses logs são enviados ao Application Insights automaticamente

Consultar via Kusto Query no Application Insights:

```
traces
| where customDimensions.Category startswith "Function"
| where timestamp > ago(7d)
| order by timestamp desc
```

Limpeza periódica

O botão **Limpar Base** em Configurações (admin only) está disponível para resetar a base em situações excepcionais (testes pré-produção, migração de ambiente). Pasta Notas Aprovadas é preservada por padrão (contém histórico legacy da Pronep).

Parte V — Anexos

19. Checklist de implantação

Fase 0 — Preparação (1 dia)

- ☐ Solicitar acesso Azure (Subscription com Contributor)
- ☐ Solicitar acesso Entra ID (Application Administrator + Groups Administrator)
- ☐ Solicitar acesso SharePoint Admin
- ☐ Solicitar acesso Teams Admin Center
- ☐ Solicitar acesso ao repositório GitHub (Admin)
- ☐ Identificar a conta de envio de e-mails
- ☐ Definir nome do Static Web App (aprovacao-nf-<unidade>)
- ☐ Definir URL do site SharePoint

Fase 1 — Entra ID (2-3 horas)

- ☐ Criar App Registration
- ☐ Anotar Client ID + Tenant ID
- ☐ Criar Client Secret e salvar em cofre
- ☐ Habilitar ID tokens em Authentication
- ☐ Adicionar groups claim em Token configuration
- ☐ Adicionar 7 permissões Graph + admin consent
- ☐ (Opcional) Configurar Expose an API para Teams SSO
- ☐ Criar 13 grupos de segurança
- ☐ Adicionar membros iniciais nos grupos
- ☐ Anotar todos os 13 GUIDs

Fase 2 — SharePoint (3-4 horas)

- ☐ Criar site (se necessário)
- ☐ Criar lista PRONEP-NF-NotasFiscais com 22 colunas
- ☐ Criar lista PRONEP-NF-Fornecedores com 11 colunas
- ☐ Criar lista PRONEP-NF-Diretorias com 5 colunas
- ☐ Popular Diretorias com 27 linhas (3 unidades × 9 diretorias)
- ☐ Criar lista PRONEP-NF-Config e adicionar item global
- ☐ Criar lista PRONEP-NF-PushSubscriptions (se for usar push)
- ☐ Criar estrutura de pastas em Notas Fiscais/
- ☐ Validar tipos de colunas (Moeda, Hyperlink, Choice)

Fase 3 — Código (2-3 horas)

- ☐ Clonar repositório
- ☐ Substituir GUIDs em api/MeusGrupos/index.js
- ☐ Substituir ADMIN_GROUP_ID em api/AdminLimparBase/index.js
- ☐ Atualizar teams-app/manifest.json (URL, ID, webApplicationInfo)
- ☐ Commit + push na branch main

Fase 4 — Azure SWA (1-2 horas)

- ☐ Criar Static Web App (Standard SKU) conectado ao GitHub
- ☐ Aguardar primeiro deploy automático ficar verde
- ☐ Configurar Redirect URI na App Reg (após ter URL pública do SWA)
- ☐ Adicionar 6 App Settings obrigatórias
- ☐ (Opcional) Adicionar 6 App Settings opcionais
- ☐ Restart do SWA

Fase 5 — Teams (1 hora)

- ☐ Empacotar teams-app/ em ZIP
- ☐ Fazer upload no Teams Admin Center
- ☐ Marcar app como Allowed
- ☐ Configurar Setup policy para permitir auto-instalação
- ☐ (Opcional) Configurar Teams SSO completo (Expose API + Authorized client apps)

Fase 6 — Validação (2-3 horas)

- ☐ Acessar URL do SWA
- ☐ Fazer login com conta admin
- ☐ Verificar que MeusGrupos retorna a role esperada
- ☐ Cadastrar 1 fornecedor de teste
- ☐ Lançar 1 NF de teste
- ☐ Confirmar que aprovador recebeu e-mail
- ☐ Aprovar pela UI
- ☐ Confirmar watermark no PDF
- ☐ Confirmar movimentação para Notas Aprovadas
- ☐ (Opcional) Testar Push notification
- ☐ (Opcional) Testar Personal Tab no Teams

Fase 7 — Go-live (1 dia)

- ☐ Comunicar usuários
- ☐ Distribuir manual de uso
- ☐ Acompanhar lançamentos do primeiro dia
- ☐ Estar disponível pra suporte

Tempo total estimado: 5 a 7 dias úteis com 1 pessoa dedicada e acessos garantidos.

20. Troubleshooting

Login redireciona ao Microsoft mas retorna com 403 "Sem acesso"

Causa: usuário não pertence a nenhum dos 13 grupos do sistema.

Solução: adicionar o usuário em pelo menos um grupo (ex.: PRONEP-NF-Admin para teste).

`/api/MeusGrupos` retorna `roles: []`

Causa: token não está emitindo a claim groups.

Solução: voltar em Entra ID → App Reg → **Token configuration** → confirmar que a claim "groups" está habilitada com tipo "Group ID".

`/api/PostNota` retorna 500 com "Lista PRONEP-NF-NotasFiscais nao encontrada"

Causa: SHAREPOINT_SITE_HOSTNAME ou SHAREPOINT_SITE_PATH errados, ou lista com nome diferente.

Solução: confirmar que o displayName da lista é exatamente PRONEP-NF-NotasFiscais (case-sensitive). Confirmar que SHAREPOINT_SITE_PATH começa com /sites/ e não termina com /.

E-mails não estão chegando

Causa 1: EMAIL_FROM_ADDRESS não tem licença com Exchange Online.

Causa 2: Permissão Mail.Send (Application) não foi consentida pelo admin.

Solução: verificar a conta no Microsoft 365 admin, e voltar em Entra ID → App Reg → API permissions → Mail.Send → garantir status Granted.

Notificações Teams não chegam

Causa 1: usuário não tem o Teams App instalado e o auto-install via Graph falhou (política do tenant bloqueia).

Causa 2: permissões TeamsActivity.Send, TeamsAppInstallation.ReadWriteForUser.All, AppCatalog.Read.All não foram concedidas.

Solução: verificar permissions na App Reg, e verificar a Setup Policy de Teams Apps no Admin Center.

Push Notifications não funcionam

Causa: VAPID keys ausentes ou inválidas.

Solução: gerar VAPID keys (seção 9.1), adicionar no Azure SWA Configuration, restart.

PDF errado aparece quando clico em "Abrir PDF" nas Notas Aprovadas

Causa: histórico — versão antiga do AbrirPdfDaNota tinha fallback "mais recente" que retornava PDF aleatório quando não achava match exato. Já foi corrigido — match exato pelo número da NF.

Solução: garantir que está rodando a versão atualizada do código (commit pós-fix).

Deploy do GitHub Actions falha com "401 Unauthorized" ou "Bad credentials"

Causa: token AZURE_STATIC_WEB_APPS_API_TOKEN invalidado ou expirado.

Solução: Portal Azure → SWA → Overview → **Manage deployment token** → Reset → copiar novo → atualizar secret no GitHub.

Function nova adicionada não fica disponível (HTTP 404)

Causa possível 1: deploy não copiou a pasta (cache do build).

Causa possível 2: erro de require no index.js que quebra o startup silenciosamente.

Solução:

Bump no version do api/package.json para invalidar cache

Comparar o function.json da nova com uma que funciona

Verificar se algum require() no topo do index.js quebra (testar localmente com node -e "require('./api/<NomeFunction>/index.js')")

Listar Functions no Azure Portal → SWA → Functions

Filtros mobile estouram a largura no iOS Safari

Causa: Safari iOS às vezes ignora `display: grid !important` quando há `style="display: flex" inline` conflitante.

Solução: usar a classe `filter-stack-mobile` com `display: flex !important; flex-direction: column !important`. Já implementado no código atual.

21. Mapa de permissões Graph

Permissão (Application)	Usada por	Por quê
GroupMember.Read.All	MeusGrupos, ListarGestoresFinanceiro, AdminLimparBase	Listar grupos do usuário e membros de um grupo
User.Read.All	teamsActivity.js (resolver email → oid)	Encontrar o objectId do destinatário Teams
Sites.ReadWrite.All	Todas as Functions que tocam SharePoint	Ler e gravar nas Listas e Document Library
Mail.Send	email.js	Enviar emails transacionais
TeamsActivity.Send	teamsActivity.js	Enviar notificação 1:1 no Teams
TeamsAppInstallation.ReadWriteForUser.All	teamsActivity.js (garantirAppInstalada)	Instalar o Teams App pro user automaticamente
AppCatalog.Read.All	teamsActivity.js (getCatalogAppId)	Descobrir o catalogAppId do app

Todas com **Admin Consent** concedido.

22. Glossário

Termo	Significado
SWA	Static Web App (recurso Azure que hospeda este sistema)
App Reg / App Registration	Aplicação registrada no Entra ID que tem identidade própria
Easy Auth	Sistema nativo do Azure SWA que faz autenticação OpenID Connect transparente
Teams SSO	Single Sign-On do Microsoft Teams — permite que o app embarcado obtenha token do user sem novo login
MSAL	Microsoft Authentication Library — usada pelo Teams JS SDK
JWKS	JSON Web Key Set — endpoint público da Microsoft com chaves para validar JWTs
VAPID	Voluntary Application Server Identification — padrão Web Push baseado em chaves ECDSA P-256
PWA	Progressive Web App — aplicação web instalável que funciona como app nativo
Service Worker	Script de background do navegador que faz cache, push e interceptação de requests
RBAC	Role-Based Access Control — controle de acesso por perfil
Hash SHA-256	Função criptográfica que gera "impressão digital" única do arquivo
JWT	JSON Web Token — formato padrão de tokens assinados
Watermark	Marca d'água sobreposta no PDF (texto "APROVADO" + data + aprovador)

D+5 dias úteis	5 dias úteis a partir de hoje (considerando feriados nacionais via Brasilapi)
2º nível	Aprovação adicional exigida para NFs acima do valor limite configurado
catalogAppId	ID interno do Teams App publicado no catálogo do tenant (diferente do externalId do manifest)

FIM DO MANUAL

Pronep Life Care — Sistema de Aprovação de Notas Fiscais

Versão 1.0 — Maio/2026

Em caso de dúvidas técnicas, consultar Rafael Machado (rafael.machado@pronep.com.br) ou o time de TI/Analytics Pronep.